ЛАБОРАТОРНА РОБОТА №5

СИСТЕМНЕ ПРОГРАМУВАННЯ

ДИНАМІЧНА ІДЕНТИФІКАЦІЯ ТИПІВ

Заяць Діани Юріївни

Хід виконання роботи:

5.1. Створення Windows Forms Application

Для виконання лабораторної роботи було створено новий проєкт типу Windows Forms Application. У межах цього проєкту реалізовано графічний застосунок, який після запуску відкриває головну форму Form1. Запуск програми здійснюється через файл Program.cs, де викликається метод Application.Run(new Form1()), що ініціалізує форму та запускає цикл обробки подій Windows Forms. Це повністю відповідає вимозі методичних вказівок щодо створення настільного застосунку.

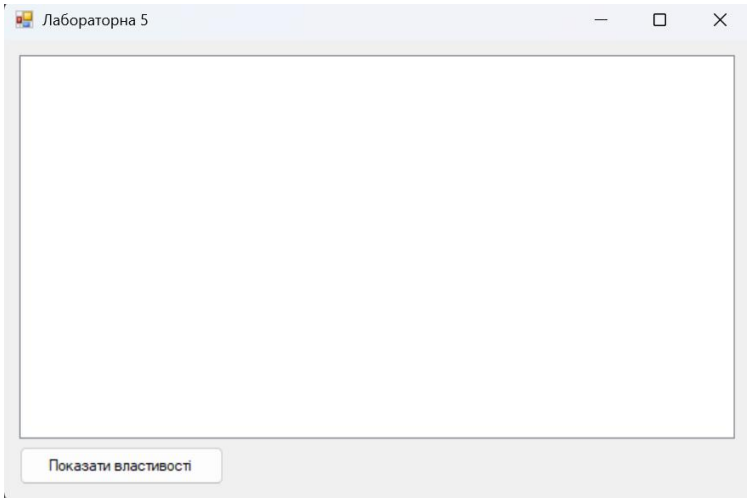

Рис. 5.1 – Запуск головної форми застосунку

5.2. Додавання на форму елемента керування TreeView

На форму було додано елемент керування TreeView з іменем treeView1. Він використовується для виведення структури даних у вигляді дерева. Також на форму додано кнопку buttonShowProperties з текстом «Показати властивості». Після натискання на цю кнопку запускається логіка створення об’єкта та виведення його властивостей у дерево. Такий підхід зручний для демонстрації

результату, оскільки заповнення TreeView відбувається не автоматично при відкритті форми, а в момент явної дії користувача. У дизайнері форми також налаштовано основні параметри інтерфейсу: розмір вікна, розміщення елементів та підписка кнопки на подію Click.

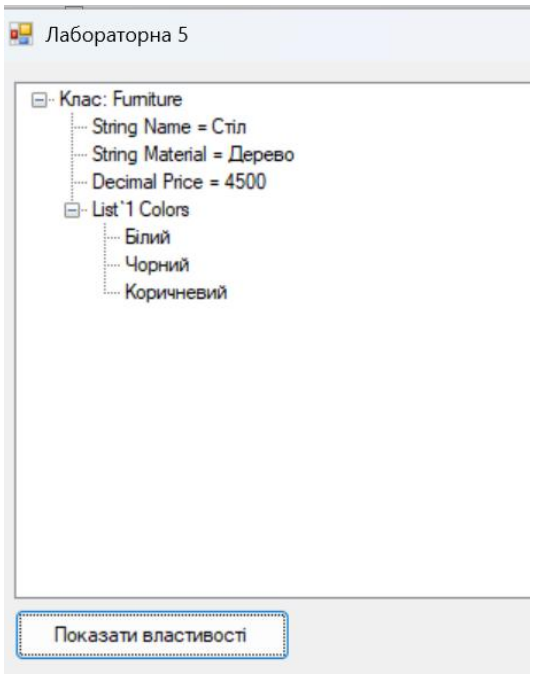

Рис. 5.2 – Інтерфейс форми з елементами TreeView і кнопкою

5.3. Опис класу предметної області

Відповідно до **варіанта 7** було реалізовано клас **Furniture («Меблі»)**. У ньому описано чотири властивості різних типів: Name типу string, Material типу string, Price типу decimal та Colors типу List<string>. Таким чином, одна з властивостей є колекцією, як і вимагалося в методичці. Крім цього, у класі реалізовано два конструктори: конструктор без параметрів, який встановлює початкові значення за замовчуванням, і конструктор з параметрами, який дозволяє одразу задати всі характеристики об’єкта. Також описано три методи: AddColor(string color) для додавання нового кольору до списку, GetColorsCount() для повернення кількості кольорів та GetInfo() для формування текстової інформації про об’єкт. Отже, структура класу повністю відповідає вимогам лабораторної роботи.

```
using System;
using System.Collections.Generic;

namespace лаб5
{
    /// варіант 7 – Меблі
    public class Furniture
    {
        // чотири властивості різних типів
        public string Name { get; set; }
        public string Material { get; set; }
        public decimal Price { get; set; }
        public List<string> Colors { get; set; }

        // конструктор без параметрів
        public Furniture()
        {
            Name = "Невідомо";
            Material = "Невідомо";
            Price = 0;
            Colors = new List<string>();
        }

        // конструктор з усіма властивостями
        public Furniture(string name, string material, decimal price, List<string> colors)
        {
            Name = name;
            Material = material;
            Price = price;
            Colors = colors ?? new List<string>();
        }

        // методи
        public void AddColor(string color)
        {
            if (Colors == null)
                Colors = new List<string>();
            Colors.Add(color);
        }

        public int GetColorsCount()
        {
            return Colors != null ? Colors.Count : 0;
        }

        public string GetInfo()
        {
            return string.Format("{0}, {1}, {2}, кольорів: {3}", Name, Material, Price, GetColorsCount());
        }
    }
}
```

Рис. 5.3 – Опис класу Furniture

5.4. Реалізація методу виведення властивостей у TreeView

Для виконання основної частини лабораторної роботи у файлі Form1.cs було реалізовано метод ShowObjectPropertiesInTreeView(object obj). Його призначення полягає в тому, щоб динамічно отримати всі публічні властивості переданого об’єкта та відобразити їх у TreeView разом із типом і значенням.

На початку роботи методу дерево очищується за допомогою treeView1.Nodes.Clear(). Далі перевіряється, чи не є переданий об’єкт порожнім. Після цього за допомогою obj.GetType() визначається фактичний тип об’єкта. На основі отриманого типу створюється кореневий вузол дерева з підписом Клас: Furniture. Потім викликається type.GetProperties(), що дозволяє отримати масив об’єктів PropertyInfo, які описують усі публічні властивості класу. Для кожної властивості окремо отримуються її ім’я, тип та поточне значення через prop.Name, prop.PropertyType.Name і prop.GetValue(obj).

Якщо властивість є звичайним значенням, вона додається до дерева як окремий дочірній вузол у форматі Тип Ім’я = Значення. Якщо ж властивість є колекцією, виконується перевірка через IEnumerable, після чого формується окремий

вузол для самої колекції, а всі її елементи додаються як вкладені дочірні вузли. Окремо враховано виняток для типу string, оскільки рядок також реалізує IEnumerable, але в цій роботі він не повинен розбиватися на символи. Наприкінці дерево розгортається методом ExpandAll(), щоб користувач одразу бачив усю структуру.

```
private void ShowObjectPropertiesInTreeView(object obj)
{
    treeView1.Nodes.Clear();
    if (obj == null)
        return;

    Type type = obj.GetType();
    TreeNode rootNode = new TreeNode("Клас: " + type.Name);
    treeView1.Nodes.Add(rootNode);

    PropertyInfo[] properties = type.GetProperties();

    foreach (PropertyInfo prop in properties)
    {
        object value = prop.GetValue(obj);
        string typeName = prop.PropertyType.Name;
        string name = prop.Name;

        if (value != null && value is IEnumerable && !(value is string))
        {
            TreeNode collectionNode = new TreeNode(string.Format("{0} {1}", typeName, name));
            foreach (object item in (IEnumerable)value)
            {
                string itemText = item != null ? item.ToString() : string.Empty;
                collectionNode.Nodes.Add(new TreeNode(itemText));
            }
            rootNode.Nodes.Add(collectionNode);
        }
        else
        {
            string valueText = value != null ? value.ToString() : string.Empty;
            string nodeText = string.Format("{0} {1} = {2}", typeName, name, valueText);
            rootNode.Nodes.Add(new TreeNode(nodeText));
        }
    }

    rootNode.ExpandAll();
}
```

Рис. 5.4 – Метод ShowObjectPropertiesInTreeView(object obj)

Демонстрація роботи програми

Для перевірки роботи програми в обробнику buttonShowProperties_Click створюється об’єкт Furniture з тестовими даними: назва — «Стіл», матеріал — «Дерево», ціна — 4500, а список кольорів містить значення «Білий», «Чорний» і «Коричневий». Після цього викликається метод ShowObjectPropertiesInTreeView(furniture), який заповнює дерево властивостями створеного об’єкта. У результаті на екрані відображається кореневий вузол класу та дочірні вузли для всіх його властивостей, включно з вкладеним списком кольорів. Це підтверджує коректну роботу механізму рефлексії та правильне відображення як простих типів, так і колекції.

Висновок

У ході виконання лабораторної роботи було створено Windows Forms застосунок для динамічного відображення властивостей об’єкта за допомогою рефлексії. Було реалізовано клас Furniture, який містить чотири властивості різних типів, дві форми конструктора та три методи. За допомогою методів GetType(), GetProperties() і GetValue() вдалося отримати інформацію про об’єкт під час виконання програми без ручного переліку його властивостей у коді. Результати роботи були відображені у компоненті TreeView, причому для колекції Colors реалізовано окреме вкладене представлення її елементів. Таким чином, мету роботи досягнуто, а всі вимоги методичних вказівок виконано.